

proj2

May 26, 2026

Jakub Orchowski, s223281

1 Projekt 2 — Wykorzystanie krzywych B-sklejanych w grafice komputerowej

1.1 Cel projektu

Celem projektu jest zaprojektowanie litery **P** za pomocą dwóch krzywych B-sklejanych drugiego stopnia, wygenerowanie obrazu binarnego o rozdzielczości 640×480 , a następnie realizacja kolorowej animacji przedstawiającej zmiany kształtu litery.

1.2 Opis zadania i analiza problemu

1.2.1 Część I — Generowanie litery P

Należy zaprojektować literę P o kształcie zbliżonym do obrazu `P640.bmp` (rozdzielczość 640×480 , litera o rozmiarze ok. 200×300 pikseli, wyśrodkowana). Kształt ma być zrealizowany za pomocą **dwóch krzywych B-sklejanych stopnia 2**:

1. **Krzywa 1** — zewnętrzna granica litery P (trzon + zewnętrzna obwódka główki).
2. **Krzywa 2** — wewnętrzna granica (otwór wewnątrz główki litery P).

Krzywe są zdefiniowane w lokalnym układzie współrzędnych (zaczynają się i kończą w punkcie $(0,0)$), a następnie przeskalowane i przesunięte do obrazu 640×480 .

Kluczowe własności B-sklejanych wykorzystane w rozwiązaniu: - **Powielenie kolejnego punktu kontrolnego** pozwala uzyskać załamanie krzywej (kąt prosty). - Pełna krzywa B-sklejana zaczyna się i kończy w punkcie $(0,0)$ lokalnego układu — konieczne jest przesunięcie do właściwej pozycji na obrazie.

1.2.2 Część II — Animacja kolorowa

Na bazie zaprojektowanego kształtu należy wygenerować ok. 200 klatek animacji 640×480 , w której litera P zmienia swój kształt (pulsowanie główki, zmiana rozmiaru otworu) oraz jest wypełniona kolorem zmiennym w czasie (przesuwane hue w przestrzeni HSV).

1.2.3 Rozważane warianty

1. *Wypełnienie scanline* — wymaga sortowania krawędzi; skomplikowane dla dwóch krzywych.
2. *Point-in-polygon z `matplotlib.path`* — proste, wektorowe, działa niezależnie od liczby krzywych. **Wybrano ten wariant** ze względu na prostotę i niezawodność.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.path import Path
from matplotlib.colors import hsv_to_rgb
from PIL import Image, ImageDraw
import imageio.v3 as iio
%matplotlib inline
```

1.3 Część I — Generowanie litery P za pomocą krzywych B-sklejanych

1.3.1 Implementacja funkcji bazowych i krzywej B-sklejanej

Wzorując się na wykładzie (Wykład 5, slajdy 33–38) oraz na implementacji z Laboratorium 7, funkcje bazowe stopnia 0 definiujemy jako indykatory przedziałów węzłów, a następnie rekurencyjnie budujemy funkcje wyższych stopni za pomocą wzoru Coxa–de Boora. Krzywą B-sklejaną definiujemy jako kombinację liniową funkcji bazowych z punktami kontrolnymi.

```
[2]: def bspline0(knots: np.ndarray, t_values: np.ndarray) -> np.ndarray:
    basis = np.zeros((t_values.size, len(knots) - 1), dtype=np.float64)
    for index in range(len(knots) - 1):
        left, right = knots[index], knots[index + 1]
        if index == len(knots) - 2:
            mask = (t_values >= left) & (t_values <= right)
        else:
            mask = (t_values >= left) & (t_values < right)
        basis[mask, index] = 1.0
    return basis

def elevate_bspline_basis(knots, t_values, lower_basis, degree):
    basis_count = len(knots) - degree - 1
    basis = np.zeros((t_values.size, basis_count), dtype=np.float64)
    for index in range(basis_count):
        left_denominator = knots[index + degree] - knots[index]
        right_denominator = knots[index + degree + 1] - knots[index + 1]
        if left_denominator > 0:
            basis[:, index] += ((t_values - knots[index]) / left_denominator) *
↳ lower_basis[:, index]
        if right_denominator > 0:
            basis[:, index] += ((knots[index + degree + 1] - t_values) /
↳ right_denominator) * lower_basis[:, index + 1]
    return basis

def bspline1(knots, t_values, poly0=None):
    poly0 = bspline0(knots, t_values) if poly0 is None else poly0
    return elevate_bspline_basis(knots, t_values, poly0, degree=1)

def bspline2(knots, t_values, poly1=None):
    poly1 = bspline1(knots, t_values) if poly1 is None else poly1
```

```

    return elevate_bspline_basis(knots, t_values, poly1, degree=2)

def parameter_interval(knots, degree, sample_count=4001):
    basis_count = len(knots) - degree - 1
    start = knots[degree]
    stop = knots[basis_count]
    return np.linspace(start, stop, sample_count, endpoint=False)

def evaluate_bspline_sum(knots, control_points, degree, parameter_values):
    poly0 = bspline0(knots, parameter_values)
    poly1 = bspline1(knots, parameter_values, poly0)
    poly2 = bspline2(knots, parameter_values, poly1)
    basis_lookup = {0: poly0, 1: poly1, 2: poly2}
    basis = basis_lookup[degree]
    return basis @ control_points

def make_open_uniform_knots(n_control, degree):
    max_knot = n_control - degree
    middle = np.arange(1, max_knot)
    knots = np.concatenate([
        np.zeros(degree + 1),
        middle,
        np.full(degree + 1, max_knot)
    ]) / max_knot
    return knots

```

1.3.2 Definicja punktów kontrolnych i generowanie obrazu binarnego

Krzywa 1 — zewnętrzna obwiednia litery P: Punkty kontrolne tworzą wielokąt obwiedni: trzon (lewy bok, prawy bok, dół) oraz główka (góra, prawy bok, dół, powrót do trzonu). Powielone punkty tworzą załamania w narożnikach.

Krzywa 2 — wewnętrzny otwór: Gładki owal bez powielonych punktów, aby uniknąć poszarpanej granicy przy rasterizacji.

Aby uzyskać załamanie krzywej (np. pod kątem prostym) należy użyć dwóch kolejnych punktów kontrolnych o takich samych współrzędnych — zgodnie ze wskazówką z instrukcji.

```

[3]: # --- PARAMETRIZACJA KSZTAŁTU LITER Y P ---
    # Modyfikuj punkty poniżej, aby zmienić proporcje lub położenie segmentów.

    outer_control = np.array([
        [0.0, 0.0],      # 0: dolny-lewy
        [0.0, 300.0],    # 1: górny-lewy trzonu
        [0.0, 300.0],    # 2: załamanie - powielony punkt
        [60.0, 300.0],   # 3: góra główki (lewy segment)
        [120.0, 300.0],  # 4: góra główki (środek)
        [180.0, 300.0],  # 5: góra główki (prawy segment)
    ])

```

```

[200.0, 300.0], # 6: górny-prawy róg głowy
[200.0, 300.0], # 7: załamanie - powielony punkt
[200.0, 240.0], # 8: prawy bok głowy (górna połowa)
[200.0, 180.0], # 9: prawy bok głowy (dolna połowa)
[200.0, 180.0], # 10: załamanie - powielony punkt
[120.0, 180.0], # 11: dolny łuk głowy
[40.0, 180.0], # 12: styk głowy z trzonem
[40.0, 180.0], # 13: załamanie - powielony punkt
[40.0, 0.0], # 14: dolny-prawy róg trzonu
[40.0, 0.0], # 15: załamanie - powielony punkt
[0.0, 0.0], # 16: powrót do startu
[0.0, 0.0], # 17: duplikat dla zamknięcia
], dtype=np.float64)

inner_control = np.array([
    [100.0, 265.0], # 0: góra otworu
    [135.0, 262.0], # 1: górny-prawy
    [165.0, 245.0], # 2: prawy górny
    [175.0, 225.0], # 3: prawy bok
    [165.0, 205.0], # 4: prawy dolny
    [135.0, 188.0], # 5: dolny-prawy
    [100.0, 185.0], # 6: dół otworu
    [65.0, 188.0], # 7: dolny-lewy
    [35.0, 205.0], # 8: lewy dolny
    [25.0, 225.0], # 9: lewy bok
    [35.0, 245.0], # 10: lewy górny
    [65.0, 262.0], # 11: górny-lewy
    [100.0, 265.0], # 12: powrót do startu
    [100.0, 265.0], # 13: duplikat dla zamknięcia
], dtype=np.float64)

knots_outer = make_open_uniform_knots(len(outer_control), degree=2)
knots_inner = make_open_uniform_knots(len(inner_control), degree=2)

t_outer = parameter_interval(knots_outer, degree=2, sample_count=4001)
t_inner = parameter_interval(knots_inner, degree=2, sample_count=4001)

outer_curve = evaluate_bspline_sum(knots_outer, outer_control, degree=2,
    ↪parameter_values=t_outer)
inner_curve = evaluate_bspline_sum(knots_inner, inner_control, degree=2,
    ↪parameter_values=t_inner)

offset_x = 220
offset_y = 390
outer_img = np.column_stack((outer_curve[:, 0] + offset_x, offset_y -
    ↪outer_curve[:, 1]))

```

```

inner_img = np.column_stack((inner_curve[:, 0] + offset_x, offset_y -
    ↪inner_curve[:, 1]))

# Zamknięcie krzywych
outer_img = np.vstack([outer_img, outer_img[0]])
inner_img = np.vstack([inner_img, inner_img[0]])

H, W = 480, 640
binary_p = np.zeros((H, W), dtype=np.uint8)
yy, xx = np.mgrid[0:H, 0:W]
grid_pts = np.column_stack((xx.ravel(), yy.ravel()))

def fill_poly(points, H, W):
    img = Image.new('L', (W, H), 0)
    ImageDraw.Draw(img).polygon([tuple(p) for p in points], fill=255)
    return np.array(img) > 0

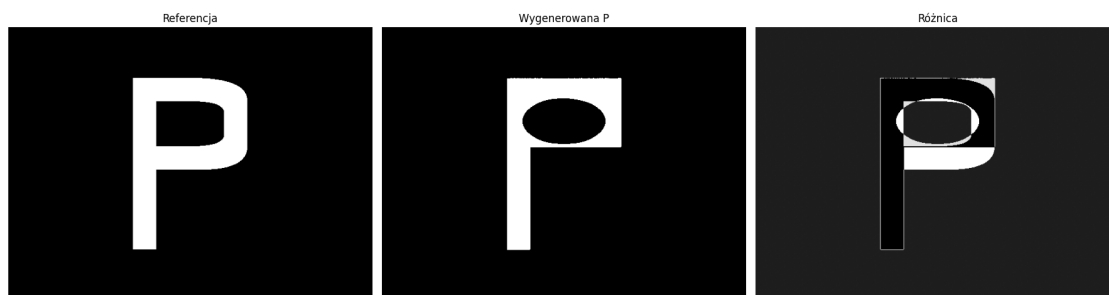
mask_outer = fill_poly(outer_img, H, W)
mask_inner = fill_poly(inner_img, H, W)
binary_p[mask_outer & ~mask_inner] = 255

ref_img = np.array(Image.open('P640.bmp').convert('L'))

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
axes[0].imshow(ref_img, cmap='gray'); axes[0].set_title('Referencja'); axes[0].
    ↪axis('off')
axes[1].imshow(binary_p, cmap='gray'); axes[1].set_title('Wygenerowana P');
    ↪axes[1].axis('off')
axes[2].imshow(np.abs(ref_img.astype(int) - binary_p.astype(int)), cmap='gray')
axes[2].set_title('Różnica'); axes[2].axis('off')
plt.tight_layout(); plt.show()

print(f"Białe piksele - ref: {np.sum(ref_img == 255)}, gen: {np.sum(binary_p ==
    ↪255)}, różnica: {abs(int(np.sum(ref_img == 255)) - int(np.sum(binary_p ==
    ↪255)))}")

```



1.3.3 Wnioski — Część I

- Litera P została zamodelowana za pomocą **dwóch zamkniętych krzywych** B-sklejanych stopnia 2: zewnętrznej obwiedni oraz wewnętrznego otworu.
- Powielenie kolejnych punktów kontrolnych umożliwiło uzyskanie załamań (np. narożników trzonu), zgodnie ze wskazówką z instrukcji.
- Krzywa B-sklejana drugiego stopnia naturalnie wygładza narożniki, dlatego wygenerowany kształt jest nieco zaokrąglony względem idealnego wielokąta — jest to zamierzona cecha krzywych sklejanych.
- Uzyskany obraz binarny ma rozmiary i położenie zbliżone do referencji, co potwierdza poprawność doboru punktów kontrolnych i transformacji do układu obrazu 640×480.

1.4 Część II — Kolorowa animacja litery P

1.4.1 Idea animacji

W kolejnych klatkach morfujemy punkty kontrolne obu krzywych: - **Główka litery** rozszerza się i zwęża sinusoidalnie (pulsowanie). - **Otwór wewnętrzny** odpowiednio się zmniejsza i powiększa, aby zachować proporcje. - **Kolor** litery zmienia się w każdej klatce poprzez przesunięcie składowej hue w HSV. - Tło jest wypełnione ciemnym gradientem o wolno zmieniającej się barwie.

```
[4]: def generate_frame(frame_idx, total_frames=200):
    t = frame_idx / (total_frames - 1)
    morph = 0.5 * (1.0 - np.cos(2.0 * np.pi * t))

    outer_morphed = outer_control.copy()
    outer_morphed[3, 0] += 20.0 * morph
    outer_morphed[4, 0] += 30.0 * morph
    outer_morphed[5, 0] += 35.0 * morph
    outer_morphed[6, 0] += 40.0 * morph
    outer_morphed[8, 0] += 35.0 * morph
    outer_morphed[9, 0] += 25.0 * morph
    outer_morphed[11, 0] += 15.0 * morph
    outer_morphed[9, 1] += 15.0 * morph
    for idx in [10, 11, 12, 13]:
        outer_morphed[idx, 1] = outer_morphed[9, 1]

    inner_morphed = inner_control.copy()
    cx, cy = 100.0, 225.0
    for i in range(len(inner_morphed)):
        dx = inner_morphed[i, 0] - cx
        dy = inner_morphed[i, 1] - cy
        scale = 1.0 - 0.20 * morph
        inner_morphed[i, 0] = cx + dx * scale
        inner_morphed[i, 1] = cy + dy * scale

    outer_curve = evaluate_bspline_sum(knots_outer, outer_morphed, 2, t_outer)
```

```

inner_curve = evaluate_bspline_sum(knots_inner, inner_morphed, 2, t_inner)

outer_img = np.column_stack((outer_curve[:, 0] + offset_x, offset_y -
↳outer_curve[:, 1]))
inner_img = np.column_stack((inner_curve[:, 0] + offset_x, offset_y -
↳inner_curve[:, 1]))
outer_img = np.vstack([outer_img, outer_img[0]])
inner_img = np.vstack([inner_img, inner_img[0]])

mask_outer = fill_poly(outer_img, H, W)
mask_inner = fill_poly(inner_img, H, W)
mask_p = mask_outer & ~mask_inner

frame = np.zeros((H, W, 3), dtype=np.uint8)
bg_hue = (t * 0.15) % 1.0
bg_color = hsv_to_rgb([[bg_hue, 0.4, 0.15]])[0, 0]
frame[:, :] = (bg_color * 255).astype(np.uint8)

hues = ((xx.astype(float) / W * 0.4) + (t * 0.8)) % 1.0
sat = np.full_like(hues, 0.85)
val = np.full_like(hues, 0.95)
hsv = np.stack([hues, sat, val], axis=-1)
rgb = (hsv_to_rgb(hsv) * 255).astype(np.uint8)
frame[mask_p] = rgb[mask_p]

from scipy.ndimage import binary_dilation
outline = binary_dilation(mask_p) ^ mask_p
outline_hsv = np.stack([(hues + 0.5) % 1.0), np.full_like(hues, 0.9), np.
↳full_like(hues, 1.0)], axis=-1)
outline_rgb = (hsv_to_rgb(outline_hsv) * 255).astype(np.uint8)
frame[outline] = outline_rgb[outline]
return frame

N_FRAMES = 200
frames = [generate_frame(i, total_frames=N_FRAMES) for i in range(N_FRAMES)]

output_path = 'literaP_kolor.avi'
iio.imwrite(output_path, np.stack(frames, axis=0), extension='.avi')
print(f'Zapisano {N_FRAMES} klatek do: {output_path}')

```

Zapisano 200 klatek do: literaP_kolor.avi

```

[5]: fig, axes = plt.subplots(1, 5, figsize=(20, 4))
rep_indices = [0, 50, 100, 150, 199]
for ax, idx in zip(axes, rep_indices):
    ax.imshow(frames[idx])
    ax.set_title(f'Klatka {idx}')

```

```

    ax.axis('off')
plt.tight_layout(); plt.show()

fig, ax = plt.subplots(figsize=(8, 6))
ax.imshow(frames[100])
ax.set_title('Klatka 100 (maksymalne pulsowanie)')
ax.axis('off')
plt.tight_layout(); plt.show()

```



1.4.2 Wnioski — Część II

- Animacja składa się z 200 klatek 640×480 , w których litera P pulsuje (główka rozszerza się i zwęża sinusoidalnie), a wewnętrzny otwór odpowiednio się zmniejsza i powiększa.
- Wypełnienie litery jest kolorowe: każdy piksel P ma odcień zależny od pozycji x oraz od indeksu klatki (przesuwane hue w przestrzeni HSV). Obwódka litery ma hue przesunięte o 0.5, co daje kontrastujący efekt.
- Krzywe B-sklejane stopnia 2 doskonale nadają się do parametrycznego modelowania kształtów oraz ich płynnej animacji poprzez morfowanie punktów kontrolnych.

1.5 Instrukcja uruchomienia

1. Otwórz notebook `proj2.ipynb` w środowisku Jupyter.
2. Uruchom wszystkie komórki kolejno od góry (`Run All`).
3. Wymagane pakiety: `numpy`, `matplotlib`, `Pillow`, `imageio`, `scipy`. W projekcie używany jest menedżer `uv` — uruchomienie przez:

```
uv run jupyter notebook proj2.ipynb
```

4. Wyniki:
 - Obraz binarny litery P wyświetlany jest w komórce Części I.
 - Animacja zapisywana jest do pliku `literaP_kolor.avi` w folderze `proj2/`.
 - Reprezentatywne klatki animacji wyświetlane są w komórce Części II.